Ενότητα 07: Ιεραρχία Μνήμης ΙΙΙ

ΕΕΙ 233: ΟΡΓΑΝΩΣΗ ΥΠΟΛΟΓΙΣΤΩΝ

ΑΝΔΡΕΑΣ ΔΙΑΒΑΣΤΟΣ

andreas.diavastos@cut.ac.cy

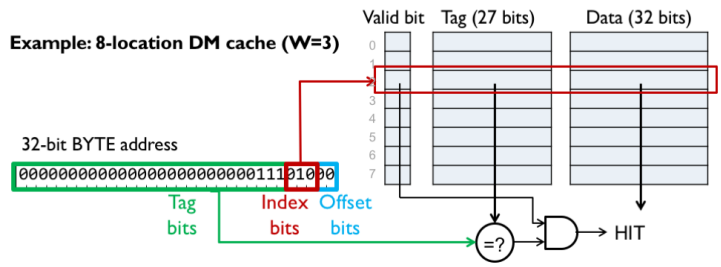
Τεχνολογικό Πανεπιστήμιο Κύπρου | Τμήμα Ηλεκτρολόγων Μηχανικών και Μηχανικών Ηλεκτρονικών Υπολογιστών και Πληροφορικής

CARL

1

Direct-mapped Caches

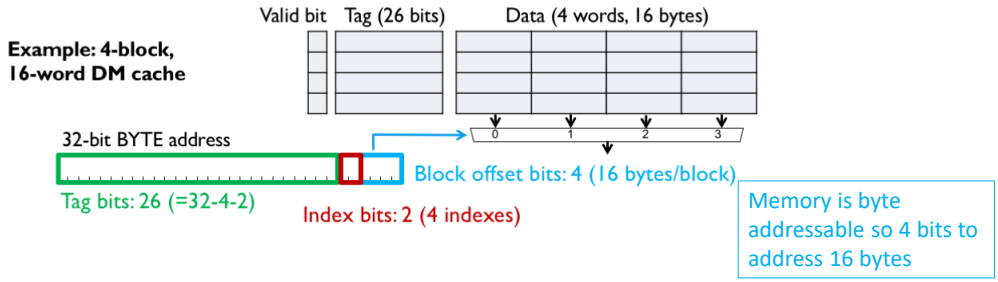
- Πρόσβαση (για cache με 2^w blocks/lines):
  - Index στη cache με w address bits (index bits)
  - Διαβάζω το Valid bit, tag και τα δεδομένα
  - Αν το Valid bit == 1 και το tag είναι το ίδιο → Cache Hit



2

Block Size

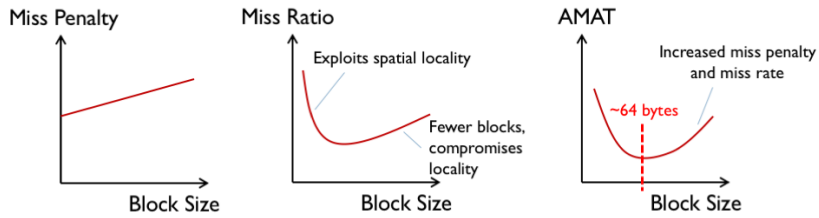
- Κάθε **block** μπορεί να κρατά περισσότερα από 1 δεδομένα
- Για να εκμεταλλευτούμε το Locality → Αυξάνουμε το Block Size
- Αυξάνοντας το Block Size → Μειώνουμε το μέγεθος του tag ή/και του Index



Block Size Tradeoffs

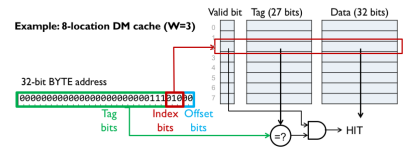


- Μεγαλύτερο Block size:
  - Εκμεταλλευόμαστε το Spatial Locality
  - Πιο λίγα blocks σε μια cache σταθερού μεγέθους
    - Περισσότερος ανταγωνισμός → ψηλότερο *Miss Rate*
  - Μεγαλύτερο *MissPenalty*
    - Παίρνει περισσότερο χρόνο να φέρουμε ένα πιο μεγάλο block στη cache
- $Average\ Memory\ Access\ Time\ (AMAT) = HitTime + MissPenalty \times MR$



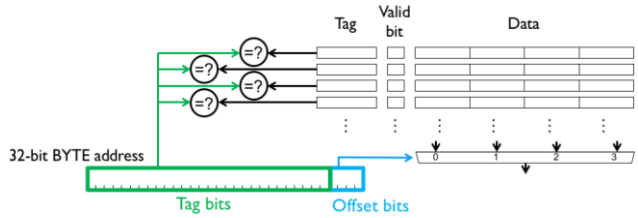
The 3 Cs model – Κατηγορίες Cache Misses

- Compulsory
  - Η πρώτη πρόσβαση σε ένα block ΔΕΝ είναι πότε στη Cache
  - Συμβαίνουν ακόμα και αν είχαμε άπειρη μνήμη
- Capacity
  - Όταν τα δεδομένα του προγράμματος που εκτελείται δεν χωράνε όλα μαζί στην cache και πρέπει να αφαιρέσουμε κάποια για να φέρουμε άλλα
- Conflict
  - Όταν πολλαπλά blocks κάνουν map στην ίδια διεύθυνση
  - Τότε πρέπει να φύγουμε κάποια δεδομένα και να τα ξαναφέρουμε όταν χρειάζονται ξανά



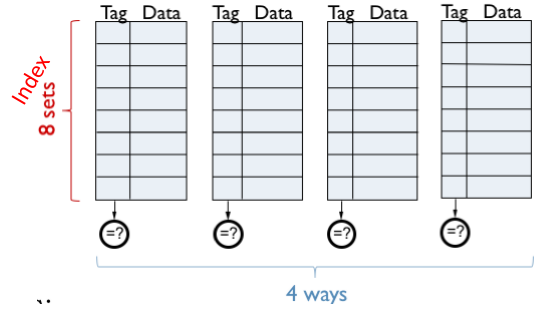
Fully-Associative Cache

- Κάθε διεύθυνση (συνεπώς και block) μπορεί να αποθηκευτεί οπουδήποτε στη cache
  - Δεν έχουμε Index
  - Ευέλικτο (ΌΧΙ conflict misses)
  - Ακριβό (Πρέπει να ελέγξουμε όλα τα tags παράλληλα για να βρούμε τη σωστή θέση – Hardware CAM technologies – Content Addressable Memory)



N-way Set Associative

- Συμβιβασμός μεταξύ Direct-mapped και Fully Associative
- Μια N-Way cache → N direct-mapped caches παράλληλα
- Direct-mapped cache → 1-Way Associative cache

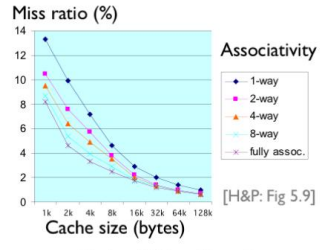


Σε 4-way set associative, για κάθε Index (block) μπορώ να το αποθηκεύσω στη cache σε 4 διαφορετικά σημεία

Associativity Tradeoffs

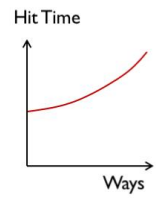
- Περισσότερα Ways
  - Μειώνουν τα Conflict Misses
  - Αυξάνουν το Hit Time

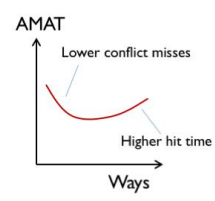
$$AMAT = HitTime + MissRatio \times MissPenalty$$



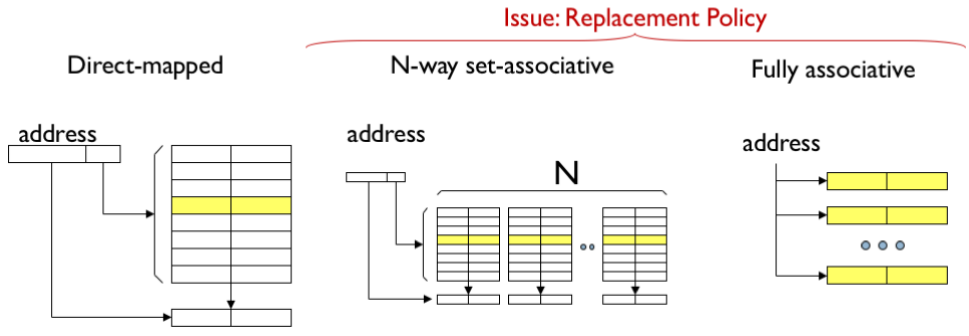
[H&P: Fig 5.9]

Little additional benefits beyond 4 to 8 ways





Που θα τα αποθηκεύσω?



Μόνο σε μία θέση στη Cache

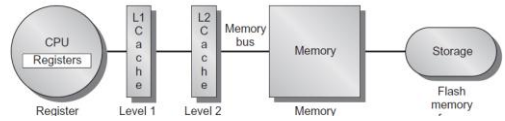
Σε N θέσεις (όσες είναι τα ways)

Σε οποιαδήποτε θέση

Replacement Policies

- Καλύτερη Λύση: Belady's MIN
  - Αντικαθιστούμε το block το οποίο θα χρησιμοποιηθεί ξανά πιο μακριά στο μέλλον
  - Πρέπει να ξέρουμε το μέλλον
  - Προβλέπουμε το μέλλον, βλέποντας το παρελθόν!
    - Εάν ένα block δεν έχει χρησιμοποιηθεί πρόσφατα, είναι πιθανόν να μην χρησιμοποιηθεί στο άμεσο μέλλον
- Least Recently Used (LRU):
  - Αντικαθιστούμε το block το οποίο χρησιμοποιήθηκε πιο μακριά στο παρελθόν
  - Καλή λύση! Πολυπλοκότητα: $O(N\log_2 N)$
  - Χρειάζεται LRU bits + logic
- Άλλες λύσεις:
  - FIFO (First In, First Out)
  - Random

Write Policy



- **Write-through:**
  - Τα writes (stores) του επεξεργαστή γράφονται στην cache και στην Κύρια Μνήμη άμεσα!
  - Σταματά τον επεξεργαστή μέχρι να ολοκληρωθεί το Write (Store)
  - Απλό, αργό και καταναλώνει αχρείαστο bandwidth
- **Write-back:**
  - Τα writes (stores) του επεξεργαστή γράφονται στην cache αλλά δεν γράφονται στην Κύρια Μνήμη μέχρι να χρειαστεί να αφαιρέσουμε το block από την cache
  - Γρηγορότερο, δεν καταναλώνει αχρείαστο bandwidth, πολύπλοκο

Cache Optimizations

Technique	Hit time	Band-width	Miss penalty	Miss rate	Power consumption	Hardware cost/ complexity	Comment
Small and simple caches	+			–	+	0	Trivial; widely used
Way-predicting caches	+				+	1	Used in Pentium 4
Pipelined & banked caches	–	+				1	Widely used
Nonblocking caches		+	+			3	Widely used
Critical word first and early restart			+			2	Widely used
Merging write buffer			+			1	Widely used with write through
Compiler techniques to reduce cache misses				+		0	Software is a challenge, but many compilers handle common linear algebra calculations
Hardware prefetching of instructions and data			+	+	–	2 instr., 3 data	Most provide prefetch instructions; modern high-end processors also automatically prefetch in hardware
Compiler-controlled prefetching			+	+		3	Needs nonblocking cache; possible instruction overhead; in many CPUs
HBM as additional level of cache		+/–	–	+	+	3	Depends on new packaging technology. Effects depend heavily on hit rate improvements

Ενότητα 07:
Ιεραρχία Μνήμης III

ΕΕΙ 233: ΟΡΓΑΝΩΣΗ ΥΠΟΛΟΓΙΣΤΩΝ

ΑΝΔΡΕΑΣ ΔΙΑΒΑΣΤΟΣ

andreas.diavastos@cut.ac.cy

Τεχνολογικό Πανεπιστήμιο Κύπρου
Τμήμα Ηλεκτρολόγων Μηχανικών και Μηχανικών Ηλεκτρονικών Υπολογιστών και Πληροφορικής

CARL

13